LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 6 INHERITANCE



NANDA MAYLA ADIASTI

103132400003

S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. Pendahuluan & Konsep Dasar Pewarisan (Inheritance)

Dalam Pemrograman Berorientasi Objek (PBO), objek-objek di dunia nyata seringkali tersusun secara hierarki. Konsep Inheritance (Pewarisan) adalah suatu proses di mana sebuah kelas dapat diturunkan dari kelas lainnya sehingga ia dapat mewarisi seluruh ciri khas, atribut (state), maupun perilaku (behaviour) dari kelas tersebut.

Latar belakang digunakannya inheritance adalah untuk menghindari duplikasi kode (code duplication). Dalam hubungan pewarisan ini, terdapat dua istilah penting:

a. Superclass (Base Class) : Kelas induk yang menurunkan sifat atau karakteristik
b. Subclass (Derived Class): Kelas anak yang diturunkan dan bersifat lebih spesifik. Hubungan ini sering disebut sebagai hubungan "is-a" (contoh: Pria is-a Manusia).

2. Keyword

Keyword super digunakan di dalam kelas turunan (subclass) untuk mereferensikan atau mengakses member dari kelas induknya (superclass), baik berupa atribut maupun method. Penggunaan super sangat berguna ketika:

a. Memanggil constructor dari superclass di dalam constructor subclass.
b. Mengakses atribut atau memanggil method milik superclass yang namanya tertutup (shadowed atau overridden) oleh subclass.

3. Method Overriding

Method Overriding terjadi ketika sebuah kelas turunan (subclass) mendefinisikan ulang sebuah method yang sudah ada di kelas induknya (superclass) dengan nama, return type, dan parameter yang sama persis.

a. Jika sebuah objek dari subclass memanggil method tersebut, maka method milik subclass-lah yang akan dieksekusi, bukan lagi method milik superclass-nya.

4. Polimorfisme (Polymorphism)

Polimorfisme adalah kemampuan suatu objek untuk mengambil berbagai bentuk yang berbeda. Konsep ini erat kaitannya dengan inheritance dan method overriding, yang memungkinkan penggunaan antarmuka (interface atau referensi superclass) yang sama untuk memerintah objek melakukan aksi yang secara prinsip seragam namun dapat memiliki bentuk proses implementasi yang spesifik pada masing-masing kelas turunannya.

B. Guided

1. Guided 1

[A.java]

```
package guided01;

public class A {
    int x;
    int y;

    void tampilXY(){
        System.out.println("Nilai x = "+x+", y = "+y);
    }
}
```

[B.java]

```
package guided01;

public class B extends A{
    int z;
    int hasil;

    void jumlahXYZ(){
        hasil = (x+y+z);

        System.out.println("Jumlah (x+y+z) = " + hasil);
    }

}
```

[Demo.java]

```
package guided01;

public class Demo {
    public static void main(String[] args){

        A superclass = new A();
        B subclass = new B();

        superclass.x = 3;
        superclass.y = 4;

        System.out.println("Superclass: ");
        superclass.tampilXY();

        subclass.x = 1;
        subclass.y = 2;
        subclass.z = 5;
```

```
        System.out.println("Subclass: ");
        subclass.jumlahXYZ();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul06 ---
Superclass:
Nilai x = 3, y = 4
Subclass:
Jumlah (x+y+z) = 8
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.816 s
Finished at: 2026-10-20T13:11:34+07:00
------------------------------------------------------------------------
```

2. Guided 2

[Employee.java]

```
package guided02;

public class Employee {
    private String nama;
    String departemen;

    public Employee(String nama) {
        this.nama = nama;
    }

    public void tampilNama() {
        System.out.println("Nama : " + nama);
    }
}
```

[Manager.java]

```
package guided02;

public class Manager extends Employee{
    private String alamat;

    public Manager(String nama, String alamat){
        super(nama);
        this.alamat = alamat;
    }

    public void tampilInformasi(){
        super.tampilNama();
        System.out.println("alamat: " + alamat);
        System.out.println("departemen: " + super.departemen);
    }


}
```

[Demo.java]

```
package guided02;

public class Demo {
    public static void main(String[] args){
        Manager mayla = new Manager("Mayla", "Menteng");
        mayla.departemen = "SDM";
        mayla.tampilInformasi();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul06 ---
Nama : Mayla
alamat: Menteng
departemen: SDM
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.868 s
Finished at: 2026-10-20T13:38:27+07:00
------------------------------------------------------------------------
```

3. Guided 3

[Lagu.java]

```
package guided03;

public class Lagu {
    String pencipta;
    String judul;

    void IsiParam(String param1) {
        judul = param1;
        pencipta = "Tidak dikenal";
    }

    void IsiParam(String param1, String param2) {
        judul = param1;
        pencipta = param2;
    }

    void tampilkan() {
        System.out.println("Judul : " + judul + ", pencipta : " + pencipta);

    }
}
```

[Main.java]

```
package guided03;

public class Main {
    public static void main(String[] args){
        Lagu d,e;
        d = new Lagu();
        e = new Lagu();

        d.IsiParam("Lagu 1");
        e.IsiParam("Lagu 2", "aku");

        d.tampilkan();
        e.tampilkan();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul06 ---
Judul : Lagu 1, pencipta : Tidak dikenal
Judul : Lagu 2, pencipta : aku
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.684 s
Finished at: 2026-10-20T13:53:08+07:00
------------------------------------------------------------------------
```

4. Guided 4

[A.java]

```
package guided04;

public class A {
    public void tampilkanKeLayar() {
    System.out.println("Method milik class A dipanggil...");
    }
 }
```

[B.java]

```
package guided04;

public class B extends A {
   public void tampilkanKeLayar() {
        super.tampilkanKeLayar();
        System.out.println("Method milik class B dipanggil...");
    }
}
```

[Main.java]

```
package guided04;

public class Main {
    public static void main(String[] args) {
        B subOb = new B();
        subOb.tampilkanKeLayar();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul06 ---
Method milik class A dipanggil...
Method milik class B dipanggil...
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.897 s
Finished at: 2026-10-20T14:03:51+07:00
------------------------------------------------------------------------
```

C. Unguided

1. Soal Unguided

Sebuah tim machine learning ingin membuat program untuk membandingkan estimasi biaya komputasi dari beberapa jenis model yang akan dilatih. Ada tiga jenis model: model regresi, model klasifikasi, dan model klasifikasi berbasis deep learning yang merupakan pengembangan dari model klasifikasi biasa.

Buatlah program Java dengan ketentuan: Penamaan kelas, atribut, dan method ditentukan sendiri oleh Anda dan semua atribut bersifat private.

[ModelML.java]

```
package unguided;

public class ModelML {
    private String namaModel;
    private int jumlahDataLatih;

    // Constructor
    public ModelML(String namaModel, int jumlahDataLatih) {
        this.namaModel = namaModel;
        this.jumlahDataLatih = jumlahDataLatih;
    }

    // Getter untuk membaca jumlah data latih
    public int getJumlahDataLatih() {
        return jumlahDataLatih;
    }

    // Getter untuk membaca nama model
    public String getNamaModel() {
        return namaModel;
    }

    // Method untuk menampilkan informasi
    public void tampilkanInformasi() {
        System.out.println("Model: " + namaModel + " | Data latih: " + jumlahDataLatih);
    }

    // Method menghitung biaya komputasi 1 (Tanpa parameter)
    public int hitungBiaya() {
        return jumlahDataLatih * 2;
    }

    // Method menghitung biaya komputasi 2 (Overloading dengan 1 parameter: epoch)
    public int hitungBiaya(int epoch) {
        return jumlahDataLatih * epoch;
    }
}
```

[ModelRegresi.java]

```
package unguided;

public class ModelRegresi extends ModelML {
    private int jumlahFitur;

    // Constructor
    public ModelRegresi(String namaModel, int jumlahDataLatih, int jumlahFitur) {
        super(namaModel, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
    }

    public int getJumlahFitur() {
        return jumlahFitur;
    }

    // Override tampilkan informasi
    @Override
    public void tampilkanInformasi() {
        super.tampilkanInformasi();
        System.out.println("Jumlah fitur: " + jumlahFitur);
    }

    // Override hitung biaya tanpa parameter
    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahFitur * 10);
    }

    // Method tambahan dengan 2 parameter (epoch dan ukuran batch)
    public int hitungBiaya(int epoch, int ukuranBatch) {
        return getJumlahDataLatih() * epoch * ukuranBatch;
    }
}
```

[ModelKlasifikasi.java]

```
package unguided;

public class ModelKlasifikasi extends ModelML {
    private int jumlahKelas;

    // Constructor
    public ModelKlasifikasi(String namaModel, int jumlahDataLatih, int jumlahKelas) {
        super(namaModel, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }

    public int getJumlahKelas() {
        return jumlahKelas;
    }

    // Override tampilkan informasi
    @Override
    public void tampilkanInformasi() {
        super.tampilkanInformasi();
        System.out.println("Jumlah kelas: " + jumlahKelas);
    }

    // Override hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) * jumlah
kelas
    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() * jumlahKelas;
    }
}
```

[ModelDeepLearning.java]

```
package unguided;

public class ModelDeepLearning extends ModelKlasifikasi {
    private int jumlahLayer;

    // Constructor
    public ModelDeepLearning(String namaModel, int jumlahDataLatih, int jumlahKelas,
int jumlahLayer) {
        super(namaModel, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    public int getJumlahLayer() {
        return jumlahLayer;
    }

    // Override tampilkan informasi dari kelas di atasnya
    @Override
    public void tampilkanInformasi() {
        super.tampilkanInformasi();
        System.out.println("Jumlah layer: " + jumlahLayer);
    }

    // Override hitung biaya tanpa parameter: biaya = (biaya versi kelas di atasnya) +
(jumlah layer x 200)
    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahLayer * 200);
    }
}
```

[Main.java]

```
package unguided;

public class Main {
    public static void main(String[] args) {
        // Membuat array bertipe kelas induk yang berisi 3 objek turunan
        ModelML[] daftarModel = new ModelML[3];
        daftarModel[0] = new ModelRegresi("Regresi Linier", 500, 4);
        daftarModel[1] = new ModelKlasifikasi("Decision Tree", 800, 3);
        daftarModel[2] = new ModelDeepLearning("Neural Network", 1000, 5, 3);

        int totalBiaya = 0;

        // Perulangan untuk menampilkan informasi, biaya komputasi, dan menjumlahkan
total biaya
        for (ModelML model : daftarModel) {
            model.tampilkanInformasi();
            int biaya = model.hitungBiaya(); // Memanggil method hitungBiaya() tanpa
parameter
            System.out.println("Biaya komputasi: " + biaya);
            totalBiaya += biaya;
        }

        System.out.println("Total biaya: " + totalBiaya + ".");

        // Menampilkan pengujian method dengan parameter khusus
        // 1. Biaya 10 epoch untuk model "Neural Network" (indeks ke-2)
        System.out.println("Biaya 10 epoch (Neural Network): " +
daftarModel[2].hitungBiaya(10));

        // 2. Biaya 10 epoch untuk model "Regresi Linier" (indeks ke-0)
        System.out.println("Biaya 10 epoch (Regresi Linier): " +
daftarModel[0].hitungBiaya(10));

        // 3. Biaya 10 epoch dengan ukuran batch 50 untuk model "Regresi Linier"
        ModelRegresi regresi = (ModelRegresi) daftarModel[0];
        System.out.println("Biaya 10 epoch batch 50 (Regresi Linier): " +
regresi.hitungBiaya(10, 50));
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul06 ---
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
Total biaya: 16440.
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 250000
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.982 s
Finished at: 2026-10-22T14:53:51+07:00
------------------------------------------------------------------------
```

Penjelasan :

1. ModelML.java
   File ini bertindak sebagai kelas induk (superclass) utama yang merepresentasikan model machine learning secara umum. Di dalam kelas ini, terdapat atribut privat seperti namaModel dan jumlahDataLatih serta sebuah constructor untuk menginisialisasi nilai-nilai tersebut. Selain itu, kelas ini dilengkapi dengan getter untuk membaca data serta menerapkan konsep method overloading pada fungsi hitungBiaya()—baik tanpa parameter maupun dengan satu parameter berupa jumlah epoch
2. ModelRegresi.java
   Kelas ini merupakan turunan dari kelas induk ModelML yang secara khusus merepresentasikan model regresi dengan menambahkan atribut privat jumlahFitur. Kelas ini menggunakan constructor milik kelas induk melalui kata kunci super serta melakukan overriding pada method tampilkanInformasi() dan method hitungBiaya() tanpa parameter agar perhitungannya disesuaikan dengan penambahan jumlah fitur. Di samping itu, kelas ini juga memiliki tambahan versi overloading hitungBiaya() yang menerima dua parameter sekaligus, yaitu jumlah epoch dan ukuran batch.
3. ModelKlasifikasi.java
   File ini berisi kelas turunan kedua yang mewarisi kelas ModelML untuk menangani model klasifikasi dengan penambahan atribut privat berupa jumlahKelas. Sama seperti kelas turunan lainnya, kelas ini memanfaatkan constructor induk dan melakukan override pada method tampilkanInformasi(). Method hitungBiaya() tanpa parameter juga di-override sedemikian rupa sehingga hasil perhitungan biaya dari kelas induk dikalikan langsung dengan jumlah kelas yang ada.
4. ModelDeepLearning.java
   Kelas ini menerapkan pewarisan bertingkat (multilevel inheritance) di mana ia mewarisi sifat dari kelas ModelKlasifikasi (bukan langsung dari kelas induk paling atas) dengan menambahkan atribut privat baru yaitu jumlahLayer. Constructor kelas ini meneruskan parameter ke kelas di atasnya (ModelKlasifikasi), serta melakukan override pada method tampilkanInformasi() untuk menampilkan informasi jumlah layer. Method hitungBiaya() juga di-override dengan menambahkan biaya spesifik berdasarkan perkalian jumlah layer dengan angka tertentu ke hasil perhitungan kelas di atasnya.
5. Main.java
   File utama ini berfungsi untuk menjalankan program secara keseluruhan dengan cara mendeklarasikan sebuah array berjenis kelas induk (ModelML[]) yang menampung tiga objek berbeda, yaitu objek regresi, klasifikasi, dan deep learning. Melalui struktur perulangan (looping), program secara polimorfik menampilkan

informasi masing-masing model, menghitung biaya komputasinya, dan mengakumulasikannya menjadi total biaya keseluruhan. Terakhir, kelas ini juga menguji pemanggilan method overloading khusus untuk menampilkan skenario biaya berdasarkan epoch maupun ukuran batch tertentu

D. Kesimpulan

Berdasarkan praktikum Modul 6 mengenai Inheritance (Pewarisan) pada Pemrograman Berorientasi Objek, dapat disimpulkan bahwa:

1. Inheritance & Encapsulation
   Konsep pewarisan memungkinkan kelas turunan mewarisi atribut dan method kelas induk untuk menghindari duplikasi kode, sementara enkapsulasi dengan modifier private menjaga keamanan data melalui getter dan constructor.
2. Keyword Super
   Digunakan untuk mengakses constructor maupun method dari kelas di atasnya secara langsung.
3. Overloading & Overriding
   Overloading memungkinkan fungsi dengan nama sama tetapi parameter berbeda, sedangkan overriding memodifikasi fungsi kelas induk agar sesuai dengan rumus perhitungan spesifik pada kelas turunan (seperti perhitungan biaya komputasi model regresi, klasifikasi, dan deep learning).
4. Polimorfisme
   Memungkinkan pengelolaan objek dari berbagai kelas turunan secara dinamis menggunakan array bertipe kelas induk.